

CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY

Faculty of Technology and Engineering CSPIT-IT

Subject: Advanced Web Development Frameworks (ITUE301) | Semester: 5th

Richardson Maturity Model Evaluating Your REST API		
Week / CO / Marks	Week 4	
CO/PO:	CO2 / PO3	
1. Problem Definition		
Topics Covered	• Richardson Maturity Model (Levels 0–3) • RESTful API Design evaluation • HATEOAS awareness	
Problem Statement	• Evaluate the Task Management API built in Practical 4 against the Richardson Maturity Model. • Identify which level (0, 1, 2, or 3) the current API satisfies and justify your answer. • Improve the API to reach at least Level 2 if it does not already satisfy it. • Document the evaluation and improvement in a MATURITY.md file in your assignment repository.	
2. Lab Session Step-by-Step		
Steps	1. Study the 4 levels of the Richardson Maturity Model (self-study reference in Week 4 Theory Concepts). 2. Create a table in MATURITY.md with columns: Level Criterion Does your API satisfy this? Evidence. 3. Evaluate each existing endpoint against the Level 0–2 criteria. 4. If any endpoint fails Level 2 (e.g., wrong HTTP verb, no status codes), fix it in the code. 5. Add a HATEOAS awareness section list 2 links you would add to the task response body if targeting Level 3:<pre>{ "id": "123", "title": "Task A", "_links": { "self": "/tasks/123", "delete": "/tasks/123" } }</pre> 6. Push MATURITY.md and any corrected route files to your assignment repository.	
3. Troubleshooting Guide		
Symptom	Likely Cause	Fix
Unsure which level API is at	Mixing up Level 1 (resources) and Level 2 (verbs + status codes)	Level 1 just means separate URLs per resource. Level 2 requires correct HTTP verbs AND meaningful status codes on every endpoint.
HATEOAS feels abstract	No real implementation required at this stage	Write the JSON structure only in MATURITY.md no code implementation needed for awareness-level credit
4. Student Assignment / Post-Laboratory Work		
Tasks	• Complete MATURITY.md evaluation table with all 4 levels assessed. • Fix any Level 2 violations in the actual route code. • Add a brief paragraph explaining why most production APIs stop at Level 2.	
5. GitHub Deliverables		
Repository	assignment-w4-<rollno> (separate from main practical repository)	

Expected Contents	• MATURITY.md with completed evaluation table and HATEOAS example • Corrected route files if any Level 2 violations were found • At least one meaningful commit with a descriptive message
6. Rubrics (Assignment Evaluation Auto-graded via GitHub Actions)	
Criteria (Marks)	Description
Maturity Evaluation (4 marks)	All 4 levels assessed with correct criteria; current API level correctly identified with evidence from actual endpoints.
Level 2 Compliance (3 marks)	All endpoints use correct HTTP verbs and return meaningful status codes; violations fixed if found.
HATEOAS Awareness (3 marks)	MATURITY.md includes a valid HATEOAS JSON example with at least 2 link relations; brief justification for stopping at Level 2 provided.
Total: 10 marks	Passing Threshold: 7 / 10 (70%) Required for exam eligibility

Threat Modelling, HTTPS/TLS Awareness, and Secure Coding		
Week	Week 7	
CO/PO	CO2, CO6 / PO3, PO10	
1. Problem Definition		
Topics Covered	• STRIDE Threat Model • HTTPS and TLS overview • Secure coding input sanitization • express-mongo-sanitize	
Problem Statement	• Apply a simplified STRIDE threat model to the Task Management application and document findings. • Enable HTTPS locally using mkcert for development-time TLS awareness. • Add input sanitization to the Express backend using express-mongo-sanitize to prevent NoSQL injection. • Document all three in a SECURITY.md file in your assignment repository.	
2. Lab Session Step-by-Step		
Steps	1. STRIDE Analysis create a SECURITY.md with a table: Threat Example in your app Mitigation applied or planned. Cover all 6 STRIDE categories: Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, Elevation of Privilege. 2. Install mkcert and generate a local certificate: mkcert -install mkcert localhost 3. Configure Express to use HTTPS with the generated cert: const https = require('https'); const fs = require('fs'); const sslOptions = { key: fs.readFileSync('localhost-key.pem'), cert: fs.readFileSync('localhost.pem') }; https.createServer(sslOptions, app).listen(5443, () => { console.log('HTTPS server running on port 5443'); }); 4. Install and apply express-mongo-sanitize: npm install express-mongo-sanitize const mongoSanitize = require('express-mongo-sanitize'); app.use(mongoSanitize()); 5. Test NoSQL injection prevention send a malicious payload and confirm it is sanitized: { "email": { "\$gt": "" }, "password": { "\$gt": "" } } 6. Add a HTTPS vs HTTP comparison section in SECURITY.md explaining what TLS protects vs what JWT protects.	
3. Troubleshooting Guide		
Symptom	Likely Cause	Fix
mkcert command not found	mkcert not installed or not in PATH	Install via: choco install mkcert (Windows) or brew install mkcert (Mac); for Linux use apt or snap
Browser shows certificate warning despite mkcert	mkcert -install not run first, so root CA not trusted	Run mkcert -install before generating the certificate; restart the browser after
Sanitizer not blocking injection	mongoSanitize() placed after route definitions	Move app.use(mongoSanitize()) before all route definitions in server.js

4. Student Assignment / Post-Laboratory Work	
Tasks	• Complete the full STRIDE table with at least one real example from the Task Management app per category. • Confirm HTTPS is running on port 5443 with no browser certificate warning. • Test and document the before/after difference with and without express-mongo-sanitize.
5. GitHub Deliverables	
Repository	assignment-w7-<rollno> (separate from main practical repository)
Expected Contents	• SECURITY.md with STRIDE table and HTTPS vs JWT explanation • Updated server.js with express-mongo-sanitize applied • HTTPS configuration code committed (cert files excluded via .gitignore) • At least one commit specifically for security hardening
6. Rubrics (Assignment Evaluation Auto-graded via GitHub Actions)	
Criteria (Marks)	Description
STRIDE Analysis (4 marks)	All 6 STRIDE categories addressed with at least one app-specific example each; mitigations noted.
HTTPS Setup (3 marks)	mkcert certificates generated; Express HTTPS server runs on port 5443; browser confirms secure connection.
Input Sanitization (3 marks)	express-mongo-sanitize applied correctly; injection test payload sanitized and documented in SECURITY.md.
Total: 10 marks	Passing Threshold: 7 / 10 (70%) Required for exam eligibility

Performance Load Testing with Apache Bench														
Week	Week 9													
CO/PO	CO4 / PO3													
1. Problem Definition														
Topics Covered	- Performance Load Testing - RPS (Requests Per Second) - Latency - Error Rate													
Problem Statement	- Run load tests against the Task Management API using Apache Bench (ab). - Measure and compare RPS, average latency, and error rate for the GET /tasks endpoint with and without caching. - Document all results in a LOADTEST.md file in your assignment repository.													
2. Lab Session Step-by-Step														
Steps	1. Confirm Apache Bench is available (pre-installed on most Linux/Mac systems): ab -V On Windows: install via XAMPP or use WSL. 2. Ensure the Task Management backend is running with MongoDB connected and at least 20 task documents seeded. 3. Run a baseline load test WITHOUT caching (disable cache temporarily): ab -n 200 -c 10 http://localhost:5000/tasks Record: Requests per second, Time per request (mean), Failed requests. 4. Re-enable caching from Practical 9 and repeat the same test: ab -n 200 -c 10 http://localhost:5000/tasks 5. Record results again and create a comparison table in LOADTEST.md:<table><thead><tr><th>Metric</th><th>Without Cache</th><th>With Cache</th></tr></thead><tbody><tr><td>RPS</td><td></td><td></td></tr><tr><td>Mean Latency (ms)</td><td></td><td></td></tr><tr><td>Failed Requests</td><td></td><td></td></tr></tbody></table> 6. Add a brief analysis paragraph: what changed, why, and what the numbers mean for scalability.		Metric	Without Cache	With Cache	RPS			Mean Latency (ms)			Failed Requests		
Metric	Without Cache	With Cache												
RPS														
Mean Latency (ms)														
Failed Requests														
3. Troubleshooting Guide														
Symptom	Likely Cause	Fix												
ab command not found on Windows	Apache Bench not installed by default	Use WSL (Windows Subsystem for Linux) or install XAMPP which includes ab.exe in the bin folder												
All requests return non-2xx responses	Server not running or route path incorrect	Confirm the backend is running on port 5000 and the /tasks route is accessible via Postman first												
No visible difference between cached and uncached	Dataset too small or TTL expired during test	Seed at least 50 documents and reduce the TTL to 120 seconds to ensure the cache remains hot during the test run												
4. Student Assignment / Post-Laboratory Work														
Tasks	- Complete LOADTEST.md with filled comparison table from actual test runs. - Write a 100-word analysis of what the results mean for production scalability.													

	• Run one additional test with higher concurrency (-c 50) and note how results change.
5. GitHub Deliverables	
Repository	assignment-w9-<rollno> (separate from main practical repository)
Expected Contents	• LOADTEST.md with comparison table and analysis • Seed script used to populate test data (if applicable) • At least one commit specifically for load testing documentation
6. Rubrics (Assignment Evaluation Auto-graded via GitHub Actions)	
Criteria (Marks)	Description
Test Execution (4 marks)	Both tests (with and without cache) run with same parameters; results match the ab output shown in terminal screenshot.
Comparison Table (3 marks)	LOADTEST.md has fully filled table with RPS, latency, and error rate for both conditions.
Analysis (3 marks)	100+ word paragraph correctly explains the difference, names the reason (cache hit vs DB query), and connects to scalability.
Total: 10 marks	Passing Threshold: 7 / 10 (70%) Required for exam eligibility

Monolithic vs Microservices Architecture Analysis																	
Week / CO / Marks	Week 10																
CO/PO	CO4 / PO3																
1. Problem Definition																	
Topics Covered	• Monolithic vs Microservices Architecture • Stateless Services • Service decomposition																
Problem Statement	• Analyze the Task Management application as a monolith and propose a microservices decomposition. • Document the comparison in an ARCHITECTURE.md file. • Identify which parts of the current codebase would become independent services and explain the trade-offs.																
2. Lab Session Step-by-Step																	
Steps	1. Create ARCHITECTURE.md and draw an ASCII diagram of the current monolithic structure: Current Monolith:<pre>[React Frontend] [Express App] ├── Auth Routes ├── Task Routes ├── Cache Layer ├── Event Emitter └── [MongoDB]</pre> 2. Propose a microservices decomposition identify at least 3 services: Proposed Microservices:<pre>[React Frontend] [API Gateway] ├── Auth Service (register, login, JWT) ├── Task Service (CRUD, cache) ├── Notification Service (EventEmitter → future queue) └── [MongoDB per service or shared]</pre> 3. Create a trade-off table in ARCHITECTURE.md:<table><tr><th>Aspect</th><th>Monolith</th><th>Microservices</th></tr><tr><td>Deployment</td><td></td><td></td></tr><tr><td>Scalability</td><td></td><td></td></tr><tr><td>Complexity</td><td></td><td></td></tr><tr><td>Team Size Fit</td><td></td><td></td></tr></table> 4. Write a "Decision" paragraph: for a 5th semester student project, which is more appropriate and why? 5. Add a "Stateless Services" note explaining how JWT authentication already makes the Task Service stateless and why this matters for horizontal scaling.		Aspect	Monolith	Microservices	Deployment			Scalability			Complexity			Team Size Fit		
Aspect	Monolith	Microservices															
Deployment																	
Scalability																	
Complexity																	
Team Size Fit																	
3. Troubleshooting Guide																	
Symptom	Likely Cause	Fix															
Unsure how to split services	Services should align with business capabilities, not code files	Each service should own one domain: auth = user identity, task = task data, notification = side-effects. If two concerns share a database table, they probably belong in the same service.															

Trade-off table feels generic	Not grounded in the actual Task Management app	Reference specific features: e.g., 'scaling Task Service independently during peak usage' instead of 'better scalability in general'
4. Student Assignment / Post-Laboratory Work		
Tasks	• Complete ARCHITECTURE.md with both ASCII diagrams and filled trade-off table. • Write the Decision paragraph justifying monolith vs microservices for this specific project. • Add the Stateless Services note connecting JWT to horizontal scalability.	
5. GitHub Deliverables		
Repository	assignment-w10-<rollNo> (separate from main practical repository)	
Expected Contents	• ARCHITECTURE.md with monolith diagram, microservices proposal, trade-off table, and decision paragraph • No code changes required documentation only • At least one commit with a descriptive message	
6. Rubrics (Assignment Evaluation Auto-graded via GitHub Actions)		
Criteria (Marks)		Description
Architecture Diagrams (4 marks)		Both monolith and microservices ASCII diagrams present; at least 3 services correctly identified with their responsibilities.
Trade-off Table (3 marks)		All 4 rows filled with app-specific examples, not generic statements.
Decision + Stateless Note (3 marks)		Decision paragraph justifies choice with reasoning specific to the project; stateless note correctly links JWT to horizontal scaling.
Total: 10 marks		Passing Threshold: 7 / 10 (70%) Required for exam eligibility

Cloud Service Models and Docker Resource Limits		
Week / CO / Marks	Week 11	
CO/PO	CO5 / PO3	
1. Problem Definition		
Topics Covered	- Cloud Computing overview - IaaS, PaaS, Serverless - Docker resource limits configuration	
Problem Statement	- Document the three cloud service models (IaaS, PaaS, Serverless) in a CLOUD.md file and map the current Docker Compose setup to the closest model. - Add memory and CPU resource limits to the Docker Compose file from Practical 11. - Test and document the effect of applying resource limits.	
2. Lab Session Step-by-Step		
Steps	1. Create CLOUD.md with a three-column comparison table: IaaS PaaS Serverless. Cover: who manages what (hardware, OS, runtime, app), example providers, when to use each. 2. Add a paragraph mapping the Docker Compose setup to the closest cloud model and why. 3. Open the docker-compose.yml from Practical 11 and add resource limits to the backend service: <pre>services: backend: deploy: resources: limits: cpus: "0.5" memory: 256M reservations: memory: 128M</pre>	
	4. Run the stack with limits applied: docker-compose up --build 5. Confirm limits are applied using docker stats: docker stats 6. Document the docker stats output screenshot and add a brief note on what happens if a container exceeds its memory limit.	
3. Troubleshooting Guide		
Symptom	Likely Cause	Fix
Resource limits not reflected in docker stats	Using docker-compose v2 syntax without Swarm mode deploy.resources only applies in Swarm	For non-Swarm use, replace deploy.resources with top-level mem_limit and cpus keys under the service
Container exits immediately after adding limits	Memory limit set too low for the app to start	Increase memory limit; Node.js typically needs at least 128MB to start; monitor with docker stats before reducing further
4. Student Assignment / Post-Laboratory Work		
Tasks	- Complete CLOUD.md with comparison table and Docker Compose mapping paragraph. - Apply and test resource limits on at least the backend service. - Document docker stats output and explain what OOM (Out of Memory) kill means for reliability.	

5. GitHub Deliverables	
Repository	assignment-w11-<rollno> (separate from main practical repository)
Expected Contents	• CLOUD.md with IaaS/PaaS/Serverless comparison and Docker mapping • Updated docker-compose.yml with resource limits applied • Screenshot of docker stats output included in CLOUD.md or docs folder • At least one commit for resource limits configuration
6. Rubrics (Assignment Evaluation Auto-graded via GitHub Actions)	
Criteria (Marks)	Description
Cloud Models Comparison (4 marks)	All 3 models compared with correct provider examples and 'who manages what' breakdown; Docker Compose correctly mapped to nearest model.
Resource Limits Applied (3 marks)	docker-compose.yml updated with CPU and memory limits; stack starts successfully with limits in place.
docker stats Documentation (3 marks)	Screenshot or recorded output of docker stats showing limits; explanation of OOM kill behavior included.
Total: 10 marks	Passing Threshold: 7 / 10 (70%) Required for exam eligibility

Blue-Green Deployment Simulation with Docker Compose		
Week / CO / Marks	Week 12	
CO/PO	CO5 / PO3	
1. Problem Definition		
Topics Covered	• Blue-Green Deployment strategy • Rolling Updates awareness • Zero-downtime deployment	
Problem Statement	• Simulate a blue-green deployment by running two versions of the backend service simultaneously using Docker Compose. • Switch traffic between blue and green versions by changing the active port mapping. • Document the process and the purpose of this strategy in a DEPLOYMENT.md file.	
2. Lab Session Step-by-Step		
Steps	1. Create two versions of the backend in docker-compose.yml blue (current) and green (new): <pre>services: backend-blue: build: ./backend environment: - VERSION=blue ports: - "5000:5000" backend-green: build: ./backend environment: - VERSION=green ports: - "5001:5000"</pre> 2. Add a /version endpoint to the backend that returns the VERSION environment variable: <pre>app.get('/version', (req, res) => { res.json({ version: process.env.VERSION 'unknown' }); });</pre> 3. Start both containers: <pre>docker-compose up --build</pre> 4. Verify blue is live on port 5000 and green on port 5001: <pre>curl http://localhost:5000/version curl http://localhost:5001/version</pre> 5. Simulate a traffic switch by changing the frontend BASE_URL from port 5000 to 5001 "switching" from blue to green. 6. Document the complete process in DEPLOYMENT.md including: what changed, how downtime was avoided, and when you would roll back.	
3. Troubleshooting Guide		
Symptom	Likely Cause	Fix
Port 5000 already in use when starting second container	Previous container still running on the same port	Stop the old container first or assign different host ports (5000 for blue, 5001 for green) as shown in the Compose file above
/version returns 'unknown'	VERSION environment variable not passed through docker-compose.yml	Confirm the environment key is correctly indented under the service definition and the server reads process.env.VERSION
4. Student Assignment / Post-Laboratory Work		

Tasks	• Complete DEPLOYMENT.md documenting the full blue-green switch process. • Add a rollback section: what steps would you take if green had a critical bug after the switch? • Compare blue-green to rolling update in 3–4 sentences: when would you use each?
5. GitHub Deliverables	
Repository	assignment-w12-<rollno> (separate from main practical repository)
Expected Contents	• DEPLOYMENT.md with blue-green process documentation and rollback plan • Updated docker-compose.yml with both blue and green service definitions • Updated backend with /version endpoint • At least one commit specifically for deployment strategy simulation
6. Rubrics (Assignment Evaluation Auto-graded via GitHub Actions)	
Criteria (Marks)	Description
Blue-Green Setup (4 marks)	Both blue and green services defined in Compose; /version endpoint returns correct version for each; both containers confirmed running.
Traffic Switch Simulation (3 marks)	Switch from blue to green demonstrated via port change; before and after curl output documented in DEPLOYMENT.md.
Documentation Quality (3 marks)	DEPLOYMENT.md includes process walkthrough, rollback plan, and blue-green vs rolling update comparison.
Total: 10 marks	Passing Threshold: 7 / 10 (70%) Required for exam eligibility

AI-Assisted UI Customization and Design													
Week / CO / Marks	Week 13												
CO/PO	CO6 / PO3, PO12												
1. Problem Definition													
Topics Covered	• AI-assisted Customized Designing • Prompt Engineering for UI • Responsible AI reviewing AI output												
Problem Statement	• Use an AI API (OpenAI or Gemini) to generate UI design suggestions for the Task Management frontend. • Implement at least one AI-suggested design change in the React application. • Critically evaluate the AI suggestion document what was accepted, what was rejected, and why.												
Steps	2. Lab Session Step-by-Step												
	1. Add a new backend route /api/ai/design-suggestion that accepts an app description and returns UI design suggestions: <pre>app.post('/api/ai/design-suggestion', async (req, res) => { try { const completion = await client.chat.completions.create({ model: 'gpt-4o-mini', messages: [{ role: 'user', content: `Suggest a color scheme, font pairing, and one layout improvement for a task management web app. Return as JSON with keys:`, colors: { primary, secondary, background }, fonts: { heading, body }, layoutTip: string` }] }); res.json(JSON.parse(completion.choices[0].message.content)); } catch (err) { res.status(200).json({ fallback: true, message: 'AI unavailable' }); } });</pre> 2. On the React frontend, add a "Get AI Design Suggestion" button that calls this endpoint and displays the result. 3. Apply at least one suggestion (e.g., the color scheme) as actual CSS variables in the React app. 4. Create AIDESIGN.md with a table: <table><tr><th>AI Suggestion</th><th>Accepted?</th><th>Reason</th></tr><tr><td>Color scheme</td><td></td><td></td></tr><tr><td>Font pairing</td><td></td><td></td></tr><tr><td>Layout improvement</td><td></td><td></td></tr></table> 5. Add a Responsible AI section in AIDESIGN.md what would happen if the AI suggested an inaccessible color contrast ratio and the developer accepted it without checking?		AI Suggestion	Accepted?	Reason	Color scheme			Font pairing			Layout improvement	
AI Suggestion	Accepted?	Reason											
Color scheme													
Font pairing													
Layout improvement													
3. Troubleshooting Guide													
Symptom	Likely Cause	Fix											
AI returns malformed JSON	Model did not strictly follow JSON format in its response	Wrap JSON.parse in try/catch; if parsing fails, return the raw text and parse manually or add 'Return ONLY valid JSON, no markdown' to the prompt											

CSS variables not applying after change	Variables defined in a scoped component instead of :root	Define CSS custom properties in :root inside index.css so they apply globally across all components
4. Student Assignment / Post-Laboratory Work		
Tasks	• Complete AIDESIGN.md with filled evaluation table and Responsible AI section. • Apply at least one AI-suggested design change visibly in the React UI. • Add a before/after screenshot of the UI in AIDESIGN.md.	
5. GitHub Deliverables		
Repository	assignment-w13-<rollno> (separate from main practical repository)	
Expected Contents	• AIDESIGN.md with evaluation table and Responsible AI analysis • Updated React frontend with at least one AI-suggested design change applied • Before/after screenshots included in AIDESIGN.md or docs folder • At least one commit for AI-assisted design implementation	
6. Rubrics (Assignment Evaluation Auto-graded via GitHub Actions)		
Criteria (Marks)		Description
AI Integration for Design (4 marks)		Backend route correctly calls AI API with a design-focused prompt; response parsed and displayed in React UI.
Design Implementation (3 marks)		At least one AI suggestion implemented as actual CSS change in the React app; visible before/after difference.
Critical Evaluation (3 marks)		AIDESIGN.md table filled with acceptance/rejection reasoning; Responsible AI section identifies at least one risk of blindly accepting AI design output.
Total: 10 marks		Passing Threshold: 7 / 10 (70%) Required for exam eligibility